



Applied Computing 1&2

1.2 Programming

2026 Outcome Analysis

Project Outline

This project requires you to:

- interpret teacher-provided solution requirements
- design a solution using design tools that represent appearance and functionality
- develop a software solution using the python programming language

Assessment

This project will be assessed in accordance with the rubric provided in this document. You must submit all relevant files as evidence.

You will have approximately 8-10 periods (400-500 mins) of class time allocated to work on this project. You may also work on it outside of class for homework.

Analysis

Scenario

You work for a Melbourne design and development studio that builds custom apps for clients.

You have been approached by a sporting body client to build a scoreboard app. The app will be used to show the score and other game information on the big screen in a stadium.

Given the fast-paced nature of sport, the app needs to be easy to use and reliable so that the current score is always displayed to the spectators.

Choice

- Sport - You may choose the type of sport that your client represents (e.g. AFL, Netball, Basketball, Rugby, etc).
- Type of app – You may choose to create either a Console app or Web app using the python programming language.

Solution requirements

Core functional requirements

The app must fulfil the following requirements:

- Home and away teams
 - Display the names
 - Edit the names
- Scoring
 - Display the current score
 - Display any breakdown of the score (e.g. goals/points)
 - Increment the score for each team (e.g. goals/points)
 - Decrement the score for each team (e.g. goals/points)
- Basic Team Stats
 - Display a custom line of text that represents a basic statistic for each team
 - Edit the custom statistic
- Reset
 - Reset the team names and scores to default values
- Security
 - Require basic authentication (username and password) to access the app

Extension functional requirements

You must include **at least one** of the following extensions to achieve a **very high** scoring on the rubric:

- Storage
 - Save team names and scores to a file/DB when updated
 - Load team names and scores from a file/DB when the app starts
- Players
 - Display the names of players and their score
 - Add players
 - Remove players
 - Assign a score to a player
 - Remove a score from a player
- Game information
 - Display key game information such as fouls, interchanges, time remaining, etc
 - Add/edit this information

Non-functional requirements

- Easy to use for people without experience using electronic scoreboards
- Easy to update/maintain the code (e.g. uses object-oriented programming and reusable functions/modules)
- The solution needs to be accurate and reliable as the umpires, players and officials will rely on it for accurately scoring the games
- Use formats and conventions so the solution is visually pleasing, easy to read, and easy to understand
- Use naming conventions, commenting and internal documentation so the code is easy to read and understand

Rubric

Criteria		Very Low	Low	Medium	High	Very High
1. Solution design		Limited or no wireframes provided. Limited or no pseudocode provided.	Some wireframes are provided, which provides a small insight into the appearance of the solution. Some pseudocode is provided, which shows how some of the functional requirements will be implemented.	A range of relevant wireframes are provided using appropriate software tools. However, there are some issues with accuracy or the annotations provided. A range of relevant pseudocode is provided using some appropriate structure (e.g. BEGIN / END).	Most wireframes are relevant and provide a moderate degree of accuracy. Together they clearly show how each screen will appear and what a user can do on each screen. Most pseudocode is relevant and provides a moderate degree of accuracy. There is good use of structure, and they provide a good insight into how the functional requirements will be implemented.	All wireframes are relevant and accurate. Relevant. They include annotations that show how the screens relate to each other and what a user can do on each screen. All pseudocode is relevant and accurate. It uses appropriate structure and command terms that indicate how each functional requirement will be implemented.
Marks allocated	30	0-6	7-12	13-18	19-24	25-30

2. Solution development		Limited requirements have been solved by the solution. Limited or no use of commenting, internal documentation, and naming conventions. Limited or no evidence of testing.	Some requirements have been solved by the solution and it is able to boot and provide the user with some functionality. Some use of commenting, internal documentation, and naming conventions. Som evidence of testing provided.	Solution solves a range of requirements and can operate without significant errors. The code is formatted using indenting and naming conventions are mostly followed. Moderate use of commenting and internal documentation to explain the code. A range of testing has been performed.	Solution solves most requirements using a range of control structures and programming features. The code is efficient and uses object-oriented features or functions to avoid code repetition. Most aspects of the code are commented or use internal documentation. Most of the functionality and appearance has been tested.	Solution solves all requirements and at least one extension requirement using appropriate control structures and programming features. The code is efficient in terms of sequence, selection and repetition. It uses appropriate indenting and naming conventions. Commenting or internal documentation are widely used to make the code readable. The solution has been fully tested.
Marks allocated	70	0-14	15-28	29-42	43-56	57-70